

# The Atemporal AIEN Postulate Causal Semantics Without Clocks

M. Drake Stapleton

Version 1.1, 2026-10-02

## 1 The proposal

I propose that an AIEN computation is defined by what causally depends on what, not by when those events occur. Physical time characterizes a realization of the computation; it does not define the computation itself.

I do not mean by this that the machine runs without time. Hardware takes time. The GPU has clocks. Electrons move. Experiments consume joules and seconds. I mean that those quantities belong to the realization boundary, not to the ontology of Omega. Omega has order without clocks. FORGE has clocks without defining meaning.

Version 0.1 of this paper stated the proposal and left its formalism incomplete. Four independent reviews found the holes: the Crumblin extension definition did not enforce what it claimed, schedule invariance was stated without its hypotheses, and the novelty claim was argued against the wrong literature. Version 1.0 repairs the formalism, states the hypotheses, and narrows the claims. Version 1.1 folds in a fifth review (GPT-6 Astra), which confirmed the v1.0 repairs hold and found the remaining gaps: the schedule theorem's transfer from abstract orderings to physical executions, the fidelity (not just preregistration) of the realization projection, occurrence identity in canonical serialization, the typing of semantic identity, and an honest statement of what the experiment can and cannot establish. What remains open is stated as open.

## 2 Glossary

Readers arriving from outside the AIEN project need the following terms, used without definition in v0.1.

**AIEN.** The cognitive driver of the architecture: the component that thinks, proposes intents and hypotheses, and reasons across possibility spaces.

**Omega.** The semantic calculus: the component that defines meaning, graphs, constraints, and formal transformation programs. This paper is about Omega's semantics.

**FORGE.** The physical realizer: the component that lowers Omega programs into physical machine operations and executes them on hardware.

**AEGIS.** The verifier: checks constraints, bounds, types, and post-execution state contracts. AEGIS verifies; it does not decide what the machine may attempt.

**Crumb.** A verified program plus its verification receipt. A program without a receipt is a claim, not a crumb.

**Receipt.** The evidence that a program does what it claims: what was checked, how, when, by which verifier, against what specification.

**Crumblines.** The lineage of admitted crumbs and receipts, with the formal structure defined in Section 5.

**The Turing.** A protocol-bound measure of net held-out explanatory compression. In the formal Turing paper,  $T(M; B, D, G, P)$  compares a candidate model  $M$  against a baseline  $B$  on held-out evidence  $D$  under shared background  $G$  and measurement profile  $P$ .

### 3 The five postulates, revised

**Postulate 1** (Atemporal semantics). *Omega semantic identity contains no primitive wall-clock coordinate. Clock measurements belong to physical realizations and receipts, not to meaning. I describe Omega as atemporal, relational, and event-driven. I do not describe it as “analog”: analog hardware may eventually be one realization, but the semantics do not need it.*

**Postulate 2** (Causal order). *The primitive dependence relation is a finite directed acyclic graph  $\rightarrow$  on semantic events. I write  $a \prec b$  for reachability,  $a \rightarrow^+ b$ , and I call  $\prec$  the causal order. It is irreflexive and transitive: a strict partial order. It is not necessarily a total order. Two events may satisfy neither  $a \prec b$  nor  $b \prec a$ ; they are then causally independent, or concurrent, and there is no need to ask which happened “first.”*

I note a departure from convention: in the distributed-systems literature,  $a \prec b$  usually reads “ $a$  happened before  $b$ .” I use it to mean “ $b$  depends upon  $a$ ,” which is closer to dataflow and causal-consistency usage than to Lamport’s happened-before (which includes program order between independent operations on one process). The mathematics is the same partial order; the reading is dependence, not precedence.

**Postulate 3** (Immutable causal past). *Once an object is sealed, its causal ancestry cannot be altered. New objects may cite existing objects. Existing objects may not acquire newly invented causes. Section 5 gives this a definition that actually enforces it; v0.1’s definition did not, as the reviews showed.*

**Postulate 4** (Causal-cone admissibility). *A claim may depend only on information reachable through its declared causal past. As a slogan this is either tautological (if “depends” means “declares”) or undecidable in general (if it means actual information flow). I therefore state the enforceable version: causal-cone admissibility is a noninterference property in the sense of Goguen and Meseguer, enforced by the type system and by sandboxing with sealed inputs. An event’s observable behavior may not vary with any state outside its declared causal past. What the postulate demands of the engineering is that every channel, clock read, random draw, and environment interaction be an explicit declared input or be absent.*

**Postulate 5** (Schedule invariance). *If two physical executions realize the same semantic graph, preserve its dependency relation, and satisfy the hypotheses of Theorem 1 (deterministic opera-*

tions, causal completeness, commuting concurrent effects), then they are realizations of the same Omega computation, regardless of clock ordering, duration, concurrency, CPU/GPU placement, or scheduling.

## 4 The semantic object is a labeled causal structure

In v0.1 I wrote that “order is all the machine’s meaning consists of.” That was too strong, and it contradicted my own formalism two sections later. Two computations with identical graph topology  $A, B \rightarrow C$ , one defining  $C = A + B$  and the other  $C = A/B$ , are plainly different computations. Order is not all of meaning. The defensible claim is narrower: timing is not part of meaning, and causal order is the only temporal structure that is.

**Definition 1** (Omega program). *An Omega program is a tuple*

$$S = (V, \rightarrow, \lambda, C, E)$$

where  $V$  is a finite set of operations,  $\rightarrow$  is a directed acyclic graph of direct causal dependence,  $\lambda$  is a labeling that commits each  $v \in V$  to its operation: model or code identity, parameters, types, and input/output port roles.  $C$  is the set of constraints the realization must satisfy (resource bounds, precision requirements, sandboxing policy).  $E$  is the set of declared effects (state transformations, I/O channels, observable outputs). The causal order is  $\prec := \rightarrow^+$ . There is deliberately no clock.

The labeling  $\lambda$  answers the reviewers’ “lottery” objection: the bare poset is shared by addition, concatenation, and a lottery, but the labeled structure is not. Edge roles matter too: “left operand” and “right operand” are part of  $\lambda$ , so the  $A \leftrightarrow B$  automorphism that worried the reviewers cannot silently swap inputs.

I separate direct dependence  $\rightarrow$  from its transitive closure  $\prec$  because v0.1 conflated them. “Depends upon” naturally names the direct relation, but Postulate 2 needs transitivity, and the causal parents  $\text{Pa}(v)$  used in Section 9 must be the direct parents, which are unrecoverable from the transitive relation without transitive reduction. The DAG is primitive; the order is derived.

## 5 The Crumblin is not a timeline

The Crumb Doctrine describes the Crumblin as an append-only lineage. Under the atemporal postulate, “append-only” must be restated, because chronological order is no longer the thing being preserved:

**“Append-only” becomes causal immutability, not chronological order.**

**Definition 2** (Crumblin state). *The Crumblin state is  $\mathcal{C} = (V, \rightarrow)$  where  $V$  is the set of admitted crumbs and receipts and  $\rightarrow$  is the direct evidential-dependence DAG. The causal order is  $\prec := \rightarrow^+$ .*

**Definition 3** (Structurally admissible extension). *An admissible extension of  $\mathcal{C} = (V, \rightarrow)$  by a new receipt  $r \notin V$  is*

$$\mathcal{C}' = (V \cup \{r\}, \rightarrow')$$

where, writing  $\text{Dep}(r) \subseteq V$  for the declared dependencies of  $r$ ,

$$\rightarrow' = \rightarrow \cup \{(d, r) : d \in \text{Dep}(r)\}, \quad \text{and} \quad \prec' := \rightarrow'^+.$$

Two things in this definition repair v0.1. First,  $\prec'$  is defined as a transitive closure over a graph that remains acyclic — the old graph is acyclic and the fresh vertex  $r$  is a sink — so  $\prec'$  is a strict partial order; v0.1 merely required the restriction  $\prec'|_V = \prec$  and left transitivity unproven. Second, all newly added edges terminate at the fresh vertex  $r$ , and no edge originates at  $r$ :  $r$  is maximal with respect to  $V$ , meaning  $\neg(r \prec' v)$  for every  $v \in V$ . A new receipt can never become a cause of an already-sealed crumb.

The v0.1 definition admitted the following counterexample, found independently by two reviewers and verified programmatically. Start with  $V = \{a, b\}$ ,  $a \prec b$ . Add  $r$  with  $\text{Dep}(r) = \{a\}$ , and define  $a \prec' r$ ,  $r \prec' b$ ,  $a \prec' b$ . All three of v0.1's conditions held: the dependencies were in  $V$ , each dependency preceded  $r$ , and the restriction to  $V$  was unchanged. Yet  $\text{Past}(b)$  grew from  $\{a\}$  to  $\{a, r\}$ : the new object was inserted into the causal past of a sealed object, violating Postulate 3. Under the repaired definition this is impossible, because  $r \prec' b$  cannot be formed: no edge of  $\rightarrow'$  leaves  $r$ .

**Lemma 1** (Causal-past immutability). *For every admissible extension  $\mathcal{C}'$  of  $\mathcal{C}$  and every  $v \in V$ ,*

$$\text{Past}_{\mathcal{C}'}(v) = \text{Past}_{\mathcal{C}}(v).$$

*Proof sketch.* Any path in  $\rightarrow'$  ending at  $v \in V$  cannot pass through  $r$ , since no edge leaves  $r$ . Hence every such path lies entirely in  $\rightarrow$ , and the pasts coincide.  $\square$

Structural admissibility is not verified admission, and I do not conflate them. The rule above accepts any fresh  $r$  with  $\text{Dep}(r) \subseteq V$ , including a receipt whose claims are false: a purported receipt asserting that `return 0` satisfies “result = 1” passes structurally with empty dependencies. Admission additionally requires an independent judgment

$$\text{Check}(p, \varphi, \rho, \text{policy}) = \text{accept},$$

relating the program  $p$ , the claimed proposition  $\varphi$ , the inputs  $\rho$ , and the verifier policy. That acceptance implies the claimed semantic properties — including the noninterference of Postulate 4 over the specified observables — is an explicit architectural assumption and open obligation of AEGIS, not a theorem of this paper. The observables covered are computed values, declared effects, and output traces; timing is excluded; termination behavior must be declared. Finite testing cannot establish the universally quantified noninterference property for unrestricted programs, and I do not claim it does.

The append-only rule is therefore:

**The causal past of an admitted crumb is immutable.**

Two receipts  $A$  and  $B$  created independently satisfy neither  $A \prec B$  nor  $B \prec A$ . The database may physically write  $A$  at 12:01 and  $B$  at 12:02; another replica may receive  $B$  before  $A$ . Neither ordering means anything to Omega. Later  $C$  may depend on both. Then  $A$  and  $B$  belong to  $C$ 's causal past but remain unordered relative to one another.

This is the “sequential growth” step of causal-set dynamics (Rideout and Sorkin): each new element is added maximal with respect to the existing structure. I cite it because the causal-set literature is the closest mathematical relative of this construction, now as a precise technical borrowing rather than the loose analogy of v0.1.

**Retrospective causal inference.** A strong objection, raised in review: science routinely revises causal understanding after the fact, discovering that  $A$  caused  $B$  though both were recorded as concurrent. Does immutability forbid this? It forbids rewriting the old graph, and it should. The revision is itself a new crumb: a receipt asserting the newly discovered relation, citing both  $A$  and  $B$  as evidence, admitted maximally like any other. A paper does not rewrite the lab notebook; it cites it. The formalism therefore distinguishes the record of what was admitted when from later claims about what those admissions mean, and both are preserved.

I keep two relations distinct here, because v1.0 blurred them:  $A <_{\text{domain}} B$ , an asserted fact about the subject matter, versus  $A \prec_{\text{ledger}} B$ , a recorded evidential-dependence edge. A new receipt can assert the first; it cannot retroactively create the second. Asserted domain propositions live in the claim language  $\varphi$  of the admission judgment, not in the ledger order. No change to the extension lemma is needed, because the lemma was never about domain facts.

**Replicas.** I acknowledge what the formalism relocates rather than eliminates: each replica’s admission sequence is a local total order, a linear extension of the causal order. A global “Crumblin state” across replicas needs consistent cuts in the sense of Chandy and Lamport, and the merge of replica states is future work, not claimed here.

## 6 Schedule invariance, with hypotheses

**Theorem 1** (Schedule invariance). *Let  $(V, \rightarrow)$  be a finite DAG with causal order  $\prec := \rightarrow^+$ , labels  $\lambda$ , and a fixed external-input valuation  $\rho$ . Suppose:*

1. **Determinism.** *Each event  $v$  computes a deterministic, terminating transition on execution configurations from its declared causal inputs: its direct predecessors’ outputs, its labeled parameters  $\lambda(v)$ , and the restriction of  $\rho$  to its declared external inputs — and nothing else.*
2. **Causal completeness.** *All randomness, I/O, clock reads, mutable state, and environment interactions are explicit declared inputs, i.e. members of  $\text{Pa}(v)$  for the events they influence, or absent entirely.*
3. **Diamond condition.** *For causally incomparable enabled events  $a, b$  with semantic state transformations  $T_a, T_b$  on a configuration space containing all semantic observations, both execution orders are defined and  $T_a \circ T_b = T_b \circ T_a$ .*
4. **Result map.** *A specified map  $O$  from completed configurations to semantic results; two computations are “the same” when their  $O$ -images are equal.*

*Then every dependency-respecting serial execution of the graph yields the same semantic result  $O$ .*

*Proof sketch.* Any two linear extensions of a finite partial order can be transformed into one another by a sequence of swaps of adjacent incomparable elements. By hypothesis 3, each such swap leaves the composed transformation unchanged, and by hypotheses 1, 2, and 4 no event’s contribution depends on anything but its declared causal inputs.  $\square$

Three things about this statement are new in v1.1, and each closes a real gap. First, the configuration space contains *all* semantic observations, including declared output traces: if incomparable  $A$  and  $B$  emit fixed messages **a** and **b** to a declared channel, their store-only transformations are both

the identity and commute, yet the observable traces  $\mathbf{ab}$  and  $\mathbf{ba}$  differ. Commutation must hold on the full observable state, not on a store-only projection, or the semantics must explicitly identify traces modulo independence. Second, the theorem is over dependency-respecting *serial* executions: a topological ordering is a serial ordering of atomic events, not an arbitrary parallel execution, so non-atomic read-modify-write implementations that lose updates are outside the theorem’s scope, not counterexamples to it. Third, semantic equality here is exact. A tolerance predicate  $|x - y| \leq \varepsilon$  is not transitive and is not preserved by downstream branching, so it cannot ground the swap argument; floating-point acceptance tolerances belong to receipt acceptance, not to semantic equality. Approximate correctness with compositional error bounds is future work.

The theorem does not by itself cover physical executions. That transfer is a separate, explicit architectural obligation:

**Realization.** Every admitted physical execution must refine a dependency-respecting serial execution of the abstract transitions, preserving the observations in the configuration space. This refinement is an obligation of FORGE and the verifier, not a theorem of this paper. Errors, divergence, and resource exhaustion are inadmissible realizations unless explicitly modeled as semantic results. A failed run is evidence about the implementation or its enforcement, not a counterexample to the theorem, until the theorem’s premises and the refinement have been established for that run. I do not claim decidability merely because the hypotheses are written down.

The hypotheses are load-bearing, and v0.1’s omission of them was the second major hole. A concrete failure without hypothesis 2: give  $A$  and  $B$  an undeclared shared counter, each reading and incrementing it. The two topological schedules of  $A \rightarrow C \leftarrow B \rightarrow D$  then produce  $(A, B) = (0, 1)$  versus  $(1, 0)$ : same DAG, different result. Postulate 4 exists precisely to declare that counter inadmissible; causal completeness is therefore an essential hypothesis of the theorem, not an implementation detail. A concrete failure without hypothesis 1: GPU floating-point reductions executed in “overlapping waves” are not bitwise reproducible across schedules, so bitwise schedule invariance cannot be claimed for floating-point arithmetic without a determinism discipline stronger than the postulate provides.

The compiler theorem of v0.1 is retained in its careful bidirectional form, with the realization obligation made explicit:

same semantic causal graph  $\not\Rightarrow$  same physical execution

every dependency-respecting serial execution, under the hypotheses above  $\Rightarrow$  same Omega semantic result,

admitted physical executions must refine such a serial execution.

**Work and span.** Two quantities belong naturally to Omega rather than to FORGE. For a program with unit-cost nodes, the *work*  $W$  is the total number of operations and the *span* (causal depth)  $S$  is the length of the longest causal chain. For  $A, B \rightarrow C \rightarrow D$ :  $W = 4$ ,  $S = 3$ . Neither depends on wall-clock time or on which valid schedule was chosen, yet they say something real about realizability: one processor needs at least four unit operations, and even unbounded parallelism cannot complete the computation in fewer than three causal stages. Omega owns  $W$  and  $S$ ; FORGE owns seconds, joules, utilization, and scheduling efficiency. This is the rigorous interface between atemporal semantics and temporal realization.

## 7 The compiler, in both directions

Lowering takes Omega meaning to a clocked physical realization. Lifting, or verification, takes a physical execution trace back up to Omega evidence plus a causal graph. The compiler may produce any physical schedule consistent with  $\rightarrow$ , and the theorem above says when those schedules are semantically identical.

The semantic/realization boundary must be a defined projection, not a gesture, and its preregistration must not be confused with its fidelity. I define it as follows. Let  $R$  be a physical execution record: event-instance identifiers, observed input/output bindings, committed effects, provenance and validation evidence, plus device identifiers, timestamps, durations, energy measurements, memory addresses, and scheduling decisions. Let  $\pi$  be the realization projection fixed before the experiment:  $\pi(R)$  retains the causal graph, the labels, and the computed values, discarding exactly the physical coordinates (when, where, how long, how much energy). Preregistration prevents post-hoc changes to the projection; it does not establish fidelity. A fixed projection can be arbitrarily unfaithful — one that always returns the reference graph would let an implementation that hardcodes  $C = 14$  pass. Fidelity requires separate justification, as explicit soundness obligations:  $\pi(R).values = observedValues(R)$ , and every projected dependency edge must be justified by the execution evidence (observed bindings, not the specification text). The recovered graph is claimed as an *actual* data-dependence graph evidenced by the record; compiler transformations such as constant folding belong to the declared lowering, not to silent projection, and speculative execution ahead of inputs is inadmissible unless the lowering declares it.

Semantic identity is then defined canonically, over event occurrences rather than operation types. Two occurrences of the same labeled operation are distinct vertices: with  $x_1, x_2$  labeled  $X$  and  $y_1, y_2$  labeled  $Y$ , the wirings  $x_1 \rightarrow y_1, x_2 \rightarrow y_2$  and  $x_1 \rightarrow y_1, x_1 \rightarrow y_2$  are non-isomorphic graphs that a label-multiset serialization cannot distinguish. Canonicalization is therefore over the complete occurrence graph: the lexicographically least encoding over all vertex numberings, each edge encoded by numbered endpoints and port roles,  $\lambda$  included in full. An implementation may use any canonical-labeling algorithm computing the same canonical form. Occurrence identifiers, where used, must be immutable and schedule-independent; equal-content occurrences, especially effectful ones, are never silently deduplicated. Concurrent  $A, B$  serialize identically regardless of whether a schedule ran  $A$  first or  $B$  first, because the serialization never consults a schedule.

Identity is typed, because v1.0’s single SEMANTIC\_ID was not a well-typed function: its declared domain was  $S = (V, \rightarrow, \lambda, C, E)$  but its preimage omitted  $C, E$  and added “claimed values,” which are not components of  $S$ . I define instead

$$\begin{aligned} \text{PROGRAM\_ID}(S) &= H(\text{canon}(V, \rightarrow, \lambda, C_{\text{sem}}, E)), \\ \text{RESULT\_ID}(S, \rho, y) &= H(\text{PROGRAM\_ID}(S), \rho, y), \\ \text{CLAIM\_ID}(S, \rho, y, \varphi) &= H(\text{RESULT\_ID}(S, \rho, y), \varphi), \end{aligned}$$

where  $C_{\text{sem}}$  are the constraints that belong to meaning (precision requirements, designated observable outputs) as opposed to realization contracts (device placement, time budgets), and the split is stated per program rather than assumed. Program identity, execution-result identity, and claim identity are three different functions; hash equality is evidence of preimage equality under a collision-resistance assumption, not identity by definition.

Labels obey a typed semantic-label schema containing no references to physical receipt identity: semantic certificates are separated from timestamped audit envelopes, so two runs differing only in

when verification occurred do not differ semantically. I distinguish physical execution coordinates, excluded from semantic identity, from timestamp-valued *program data*, which is allowed as an explicit semantic input: a program computing  $y = [t > t_0]$  takes the clock reading  $t$  as declared data, and schedule invariance holds conditional on the same external-input valuation  $\rho$ . A declared source whose value changes with scheduling is not made schedule-invariant merely by being declared.

## 8 The experiment, strengthened

The v0.1 experiment had two weaknesses the reviewers identified: equal semantic IDs across schedules are near-tautological if the ID is a canonical hash of the graph (it tests the implementation’s canonicalization, not the postulate), and the design lacked negative controls. I strengthen it.

Take one Omega causal graph:  $A \rightarrow C, B \rightarrow C, C \rightarrow D$ , with deterministic labeled operations, for example  $A = 3 + 1, B = 2(5), C = A + B, D = C^2$ , expected  $A = 4, B = 10, C = 14, D = 196$ .

*Positive arm.* Execute every topological ordering for small DAGs (here exactly two:  $A, B, C, D$  and  $B, A, C, D$ ); for larger DAGs, execute randomized orderings and device placements, including GPU execution with arbitrary delays and reversed physical completion order of independent operations. Define the semantic output independently of the implementation under test: an oracle computes the expected values from the labeled graph by a separate code path. Vary the source inputs across runs ( $A, B$  drawn fresh each run) so that an implementation hardcoding  $C = 14$  cannot pass. Require: (i) every schedule’s computed values equal the oracle’s; (ii) the label-preserving equality of causal graphs holds across schedules, where equality means identical occurrence structure with identical content-addressed vertex labels and identical edge roles (the  $A \leftrightarrow B$  automorphism must not pass: feeding  $A$ ’s output into  $B$ ’s slot is a different computation and must fail); (iii) the physical receipts report their measurements accurately. I do not require receipts to differ across schedules: identical recorded durations or placements are honest outcomes of measurement resolution, and demanding predetermined differences would reject correct executions.

*Negative controls.* Run variants, each classified by the obligation it violates: a hidden clock read and undeclared mutable shared state (causal completeness); unseeded randomness (determinism); noncommuting concurrent declared effects, e.g. two events emitting different fixed messages to the same write-only channel (diamond condition — note this is a distinct failure from hidden inputs, and must be rejected as noncommutation, not mislabeled causal incompleteness); and a run with correct values but falsified input provenance or port bindings (projection soundness). The verifier must reject each for its specific stated reason rather than normalize the differences away. A schedule-invariance claim that passes only because the verifier silently discards discrepancies is worthless; the negative controls exist to catch exactly that.

A divergent result is a counterexample to the abstract claim only after the theorem’s premises and the realization refinement have been established for that run; otherwise it is evidence of an implementation or enforcement failure (a device kernel bug, an incorrect lowering, a missing environmental input). Passing runs are bounded conformance evidence for the tested graphs and schedules, not universal proof of the architectural claim. If the positive arm fails under established premises, the postulate fails in its strongest form, and I will record that.

## 9 The Turing without time, carefully

The v0.1 section on description length needs two corrections.

First, the causal factorization. The reachability order  $\prec$  does not determine direct parents: transitive reduction recovers the cover relation, not the original edges, so  $a \rightarrow b \rightarrow c$  and  $a \rightarrow b \rightarrow c$  plus  $a \rightarrow c$  share a reachability order but differ probabilistically (a noisy copy of  $A$  factorizes over the second, not the first). I therefore name the evidence-model DAG separately as  $H = (U, F)$ , with  $\text{Pa}_H(v) = \{u : (u, v) \in F\}$ , and condition on it explicitly. If the graph  $H$  is fixed and conditioned upon, the correct form is

$$L(x_U \mid H, M, G) = - \sum_{v \in U} \log_2 P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G),$$

which additionally requires the Markov assumption  $P_M(x_U \mid H, G) = \prod_v P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G)$ : the model must actually factorize over the graph, which does not follow from the graph’s existence. If instead the graph structure is itself evidence, its model probability must be encoded:

$$P_M(D \mid G) = P_M(H \mid G) \prod_v P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G).$$

The v0.1 formula silently discarded the graph’s own description length; for a graph of probability 0.1 with node conditionals product 0.25, that is 3.3219 bits of structural evidence dropped. I take the first option: the evidence graph is conditioned upon, and the notation says so. Conditioning on  $H$  means the score measures predictive compression *given* that structure; it does not reward discovering or compressing  $H$  itself, and I state that limitation rather than presenting it as a contradiction.

Second, a terminological correction the reviewers forced: “causally independent” is used in this paper in two senses. In Postulate 2 it means poset incomparability (neither  $a \prec b$  nor  $b \prec a$ ). In the probabilistic section it would naturally mean statistical or interventional independence. These are different relations, and the provenance DAG (which receipts cite which) is not in general the data-generating causal DAG: imposing ledger structure as conditional-independence assumptions is false modeling. I keep the two graphs distinct and say which one I mean wherever both appear.

Third, the motivation. For a fixed model  $p_M$ , the chain rule joint  $-\sum \log p(x_t \mid x_{<t})$  equals  $-\log p(x_1 \dots x_n)$  under any ordering; the joint is order-free. Temporal order does epistemic work only for prequential, adaptive predictors, where it keeps the code honest against peeking. I therefore state which case the Turing is: where the protocol evaluates a fixed fitted model, the timeless form loses nothing; where the protocol is prequential, the ordering is part of the anti-cheating apparatus and must be preserved as a declared causal constraint, not silently removed. The v0.1 text did not distinguish these, and the distinction matters.

With those corrections, the timeless Turing is

$$L(D \mid M, G, H) = -\log_2 P_M(D \mid G, H), \quad T(M; B, D, G, P) = [L(B \mid G) + L(D \mid B, G, H)] - [L(M \mid G) + L(D \mid M, G, H)]$$

with the evidence-model DAG  $H$  conditioned upon as above (absorbed into the shared background  $G$  wherever the protocol fixes it once for all candidates). The Turing requires evidence, not time. That claim survives the corrections unchanged.

## 10 Complexity: work and depth, stated honestly

The v0.1 text said that removing sequential steps entirely would “require new mathematics.” The reviewers corrected me: it does not. The weak reading, no wall-clock seconds, needs no change at all, because “polynomial time” in complexity theory already means a bound on computation steps, and I now state it as polynomial work  $W(n) \leq n^k$ . Three choices are independent and I keep them separate: (1) abstract versus wall-clock cost; (2) work/size versus depth; (3) uniform versus nonuniform computation. Removing a clock does not imply nonuniformity — uniform circuit families are atemporal DAG descriptions too — and  $P/poly$  changes the uniformity and advice assumptions rather than removing causal stages, so I introduce it only where advice or nonuniform adversaries are an independently intended model. The current Claim A proof uses uniform polynomial-time adversaries, and the atemporal reading does not change that.

The Discovery Gap’s work bound is two-dimensional: work  $W(n)$  and depth  $S(n)$ , the span of the longest causal chain. For unit-cost finite DAGs  $S \leq W$  always, so polynomial work already bounds depth polynomially; the separate depth bound distinguishes levels of parallelizability, such as polylogarithmic versus polynomial depth. The postulate’s own framework provides the vocabulary to say so, and v1.0’s phrase “polynomial work with exponential depth” was impossible under the paper’s own definitions — I correct it openly rather than silently.

## 11 What changes in the existing papers, as versioned revisions

I keep the seven existing AIEN papers intact; what follows are stated revisions for future versions, not silent edits.

**EXP-002.** The diffusion worlds are Brownian motion with drift,  $dX_t = \mu dt + \sqrt{2D} dW_t$  (v0.1 mislabeled this mean-reverting; mean reversion is Ornstein–Uhlenbeck). I propose replacing EXP-002 with relational and causal law discovery: inferring a hidden causal network from observations and interventions without an external clock. I withdraw the v0.1 claim that this is a “cleaner” experiment: observational data generally identify a DAG only up to Markov equivalence under standard assumptions, and temporal order is a key identifying assumption. Interventions can refine or eliminate that ambiguity — a perfect intervention on  $X$  distinguishes  $X \rightarrow Y$  from  $Y \rightarrow X$  where observations cannot — so identifiability depends on the intervention targets and the causal model’s assumptions, which the experiment specification must state. It is a different experiment that tests what AIEN claims to be, not an easier one.

**T/s.** Demoted to a physical realization metric belonging to FORGE and hardware.  $T$ ,  $T$  per observation,  $T$  per causal event, and  $T$  per causal depth live inside Omega.

**Stale receipts.** Redefined as invalidation by changed environment, superseded dependency, or changed verification policy, not mere elapsed seconds. I note the mechanism question the reviewers raised: detecting “assumptions no longer match” requires observing the mismatch, which is itself future engineering, not claimed here.

**The Turing definition.** Revised as in Section 9: conditioned graph, Markov assumption explicit, prequential versus fixed-model distinguished.

## 12 Prior art and the narrowed novelty claim

The v0.1 novelty claim was argued against the wrong literature, and its physics invocations were unanimously judged a category error by the reviewers. I correct both.

On the physics: the causal-set program recovers Lorentzian geometry from locally finite orders plus counting, not computational dependency; Page–Wootters recovers a time parameter by conditioning, not a partial order, and faces known difficulties. I demote these to motivating analogies. The postulate depends only on order theory and concurrency semantics.

On the literature: partial orders as computational semantics are the core of concurrency theory, and I now argue against it directly. Kahn process networks (1974) with the determinacy theorem; Winskel event structures; Mazurkiewicz traces; Petri nets and occurrence nets, which pair causality with a conflict relation the current Omega lacks and will need for alternative or superseded receipts; the actor model; dataflow (Dennis, Arvind, Lucid); deterministic parallelism (LVars); vector clocks (Fidge, Mattern), which are the right comparison for Section 5’s construction since Lamport clocks extend happened-before to a total order rather than defining dependence; CRDTs and causal consistency; content-addressed builds (Nix, Bazel, Unison), whose methodology is the closest existing relative of the Section 8 experiment; W3C PROV for provenance vocabulary; Chandy–Lamport snapshots for cross-replica state; hybrid logical clocks and leases as the standard answer to staleness.

Given that landscape, I state the contribution as a proposed architectural integration, not an established novelty: making causal order the semantics and epistemology of a machine intelligence architecture, enforced by the verification and receipt machinery rather than used as a coordination mechanism between processes. I separate two things the v1.0 text blurred: implementation conformance, which the experiment in Section 8 can test, and architectural novelty, which requires comparison with prior systems and which no experiment of this form can establish. A hermetic deterministic build system could pass the same positive arm; the table below states what the architecture additionally claims to supply.

| Prior system                      | What it already provides                                                    | What AIEN additionally claims                                                                                              |
|-----------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Nix / Bazel (hermetic builds)     | Dependency graphs, reproducibility, environment isolation, content identity | Provenance connected to belief admission: verified receipts as the gate for what the machine may subsequently assert or do |
| Unison                            | Content-addressed program identity                                          | Content-addressed <i>claims and evidence</i> , not just code                                                               |
| Proof-carrying code               | Machine-checkable evidence attached to executables                          | Evidence attached to the evolving world-model, with revision as new crumbs rather than recompilation                       |
| LVars / deterministic parallelism | Schedule-independent observations via monotonicity                          | Schedule-independence as the enforced epistemology of an agent that plans and acts, not just computes                      |
| Kahn networks                     | Deterministic process networks over partial orders                          | The determinacy theorem’s obligations (deterministic, complete, commuting) made into per-receipt admission checks          |

Each row’s third column is proposed, not demonstrated. If the literature already contains any of these positions in full, I will cite it and narrow further. That is an architectural claim, and it stands or falls on whether the experiment in Section 8 passes and whether the comparison survives scrutiny.

I also correct v0.1’s Lamport characterization: Lamport clocks produce a total order consistent with causality; happened-before includes program order; and Lamport 1978 explicitly took the idea from relativity’s causal cones, so the physics analogy was in Lamport, not beyond him.

## 13 Objections

**“This is just Kahn networks / Petri nets / dataflow.”** The mathematics of partial-order semantics is indeed established, and Sections 5 through 8 now argue against that literature explicitly. What I claim as new is the architectural position: causal order as the enforced epistemology of the machine, with the receipt and verification machinery as its enforcement arm, in service of machine intelligence rather than process coordination. If the experiment passes, the claim has content; if the literature already contains this exact position, I will cite it and narrow further.

**“Something still has to run first.”** Agreed, and the postulate agrees. Implementation has sequence; meaning does not contain it. The admission sequence of a replica is a local total order, a linear extension of the causal order; the formalism relocates sequence rather than eliminating it, and Section 5 says so.

**“The photon argument was wrong, so the whole thing collapses.”** The photon framing was wrong and is dropped, along with the muddled Lorentz correction. The postulate depends on order theory and concurrency semantics, which stand independently.

**“Schedule invariance is false; I have a counterexample.”** Then the counterexample either

violates one of Theorem 1’s hypotheses or the realization refinement, in which case it is a negative control the verifier must reject for its specific violated obligation — or it satisfies all of them and still diverges, in which case the postulate fails and I will record that. A divergent run is a counterexample to the abstract claim only after the premises and the refinement have been established for that run; otherwise it is evidence about the implementation. The hypotheses are now stated precisely so that this objection is decidable.

**“Science revises causal understanding; immutability forbids it.”** Immutability forbids rewriting the record, not revising understanding. The revision is a new crumb asserting  $A <_{\text{domain}} B$ , citing the old admissions as evidence; it does not and cannot create  $A <_{\text{ledger}} B$  retroactively. The notebook is never rewritten; the new paper cites it.

**“Causal-cone admissibility is tautological.”** As a slogan about declared dependencies, yes. As the noninterference property of Postulate 4, enforced by types and sandboxing, it is a checkable engineering requirement: vary everything outside the declared causal past and observe no change in behavior.

**“This contradicts the seven papers.”** It does not touch them. The seven papers stand at their versions and evidentiary statuses. This postulate is an eighth paper, and Section 11 lists its implied revisions as versioned future work, not edits.

**“An untested postulate is just philosophy.”** Fair, and Section 8 now specifies the experiment with an independent oracle, label-preserving pass criteria, and negative controls, instead of the near-tautological design of v0.1. The pass criteria are written before anyone runs it.

## 14 Status

Version 1.1. Proposed, not peer reviewed. The formalism of Sections 4 through 6 is now closed: the extension lemma is proved, the schedule-invariance theorem is stated with its hypotheses, and semantic identity has a canonical definition. The experiment is specified but not run. Five independent reviews (four outside models, one programmatic check relayed by the author) were folded into this version; their converged must-fix findings are all addressed, and the points of reviewer disagreement are recorded in the paper’s development notes rather than hidden. The seven AIEN papers are unchanged.

The line carried forward from v0.1 stands:

|                                                               |
|---------------------------------------------------------------|
| <b>The Crumblin is not a timeline. It is a causal record.</b> |
|---------------------------------------------------------------|